THAAP

Trust, Threat Hunting & Adaptive Authorization for Payments

Implementation Blueprint — What to Build End to End

Product strategy • Architecture • APIs • Data model • Trust graph • Risk engine • Agentic payments • GenAI copilot • Security • Deployment • Testing • Codex build plan

Version 1.0 — 5 October 2026

Executive decision

Build THAAP as a real-time trust and authorization control plane for human-initiated and agent-initiated money movement. It should not be another payment processor and should not be a thin fraud-score API. Its job is to answer: “Should this action be trusted enough to let money move, under what constraints, and with what evidence?”

The product thesis is supported by three converging industry signals. J.P. Morgan argues that AI-enabled B2B fraud can compromise identity or instructions before a payment enters an otherwise legitimate workflow. Mastercard’s 2026 Agent Pay Trust Framework centers agentic commerce on identity, intent, controls, trusted execution and intelligence. AP2 and Visa Trusted Agent Protocol are introducing cryptographic mechanisms to prove agent identity and user authorization. THAAP should consume and verify these proofs, enrich them with behavioral and relationship risk, enforce enterprise policy, and preserve replayable evidence. [R1][R2][R3][R4]

The product in one sentence

THAAP determines whether a human, service or AI agent is allowed to cause a financial action, using cryptographic authorization, identity and device signals, relationship history, trust-graph context, behavior, policy and fraud intelligence — and returns a deterministic decision with evidence.

What THAAP must return

ALLOW | CHALLENGE | REVIEW | DENY
+ risk_score (0–1000)
+ trust_score (0.000–1.000)
+ decision_reason_codes[]
+ required_controls[]
+ policy_version + model_version + evidence_hash
+ replay_id

Non-negotiable design principles

Principle	Implementation consequence
Deterministic money-movement gate	LLMs never directly produce ALLOW/DENY. Final decisions are policy + rules + signed evidence + bounded statistical models.
Verify intent, not only identity	A valid user, device or agent can still execute a bad instruction. Capture what was approved, limits, purpose, beneficiary and expiry.
Trust is a graph	Evaluate relationships between actor, organization, agent, device, payee, account, invoice, session, instruction and transaction.
Everything is replayable	Persist the exact evidence, feature snapshot, rules, model versions, signatures and policy used for every decision.
Least agency	Agents get narrow, time-bounded, purpose-bounded and amount-bounded authority; default to escalation when constraints are not satisfied.
Protocol adapter, not protocol competitor	Support AP2, Visa TAP and future agent-payment standards through adapters; keep THAAP’s canonical trust model protocol-neutral.
MVP before platform sprawl	Start as a modular monolith plus model worker and event bus. Split services only at scale or organizational boundaries.

What not to build first

- A card network, acquirer, issuer processor, wallet, or money transmitter.

- A generalized SIEM or generic identity provider.
- A fully autonomous fraud “agent” with direct payment execution privileges.
- A large deep-learning fraud model before you have quality labeled events.
- A dozen microservices. The first production-quality prototype should be operable by one small engineering team.
- A proprietary agent-payment protocol. Interoperate with open/industry protocols instead.

1. Research synthesis and product opportunity

1.1 J.P. Morgan: fraud attacks trust before payment

J.P. Morgan’s September 14, 2026 analysis states that AI-enabled B2B fraud increasingly inserts compromised or impersonated identities and false payee instructions into legitimate workflows. An authorized and policy-compliant payment can therefore still be fraudulent. The practical implication for THAAP is that transaction scoring is too late and too narrow; trust must be assessed at critical handoffs such as identity changes, payee changes, approval, and release. [R1]

1.2 Mastercard: identity + intent + controls + execution + intelligence

Mastercard’s September 30, 2026 trust framework describes five pillars for agentic commerce: identity, intent, controls, trusted execution and intelligence. This maps almost one-to-one to the THAAP control plane and validates a product architecture that separates proof of authority from behavioral trust and runtime policy. [R2]

1.3 AP2: mandates create cryptographic, dispute-grade evidence

AP2 v0.2 defines Checkout and Payment Mandates, human-present and human-not-present modes, role-specific verification, receipts, and dispute evidence. It explicitly requires deterministic validation even when a role is agentic. THAAP should implement AP2 verification as an adapter and normalize valid mandates into its internal AuthorizationEvidence model. [R3]

1.4 Visa Trusted Agent Protocol: agent authenticity at the HTTP boundary

Visa’s Trusted Agent Protocol lets merchants recognize approved agents using cryptographic message signatures and agent intent. THAAP should implement HTTP Message Signature verification (RFC 9421-compatible) and treat verified agent identity as one signal — never as sufficient proof that a transaction is safe. [R4]

1.5 Identity standards: phishing-resistant and continuously evaluated trust

NIST SP 800-63-4 strengthens digital identity guidance around phishing resistance, forged media/deepfakes, continuous evaluation and modern authenticators. OpenID Shared Signals/CAEP provides a model for propagating events such as session revocation, credential change and device compliance changes. THAAP should accept continuous identity-risk signals, not assume that a login remains trustworthy until logout. [R5] [R6]

1.6 Agent security: the agent itself is part of the attack surface

OWASP’s 2026 agentic security work highlights goal hijacking, tool misuse, identity/privilege abuse, memory/context poisoning, insecure inter-agent communication, cascading failures, human-agent trust exploitation and rogue agents. This means THAAP must enforce constraints outside the LLM runtime and maintain immutable execution evidence. [R7][R8]

2. Product definition

2.1 Primary users

User	Problem THAAP solves
Bank / issuer / PSP risk team	Real-time decisioning across identity, behavior, payee, device, agent and transaction context.
Enterprise treasury / AP	Prevent business-email compromise, supplier-bank-change fraud and compromised approval workflows.
Merchant / commerce platform	Differentiate trusted agents from malicious automation and enforce agent-specific purchasing policy.
AI-agent platform	Prove delegated authority and obtain narrow payment authorization without exposing raw credentials.
Fraud investigator / SOC	Reconstruct why money moved, explore graph relationships, hunt related entities and replay the original decision.
Compliance / audit	Retrieve immutable evidence of user/agent authority, policy evaluation and controls applied.

2.2 Initial use cases

1. Beneficiary / supplier bank-account change followed by payment.
2. Business email compromise or synthetic/deepfake instruction resulting in authorized push payment.
3. New device + anomalous user behavior + high-risk payee.
4. AI shopping/procurement agent with bounded spending authority.
5. Agent attempts to exceed amount, merchant, category, geography, time or velocity constraints.
6. Repeated low-value agent transactions that are individually valid but collectively suspicious.
7. Compromised session where a credential-change or revocation signal arrives after login.
8. Fraud-ring / mule detection through shared devices, accounts, identities and transaction paths.
9. Post-incident forensic replay and related-entity threat hunting.

2.3 Canonical THAAP objects

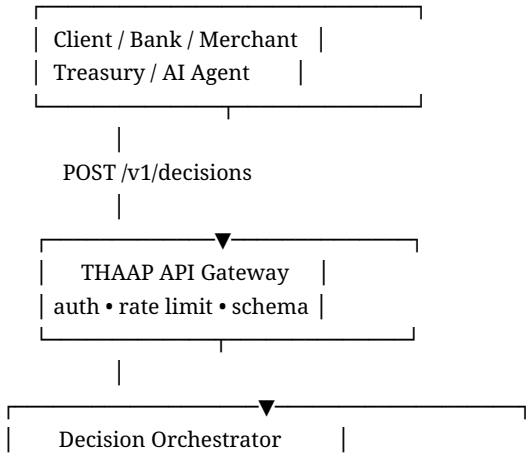
Object	Purpose	Examples
Actor	Human or legal entity initiating/owning intent	employee, customer, business
Agent	Non-human actor delegated to act	shopping agent, procurement agent
Identity	Authenticated identity + assurance	OIDC subject, passkey, certificate
Device	Observed execution endpoint	browser/device fingerprint, workload identity
Counterparty	Recipient/merchant/supplier	merchant, vendor, beneficiary
FinancialAccount	Account/token/payment instrument reference	bank account token, network token
Instruction	Requested business action	change payee, create invoice, initiate payment
Mandate	Signed authority + constraints	AP2 payment mandate, THAAP native mandate
Transaction	Proposed/attempted/executed movement	card, RTP, ACH, UPI-like push
TrustEvent	Any event changing confidence	password reset, beneficiary change, new device
Decision	Immutable authorization result	allow/challenge/review/deny

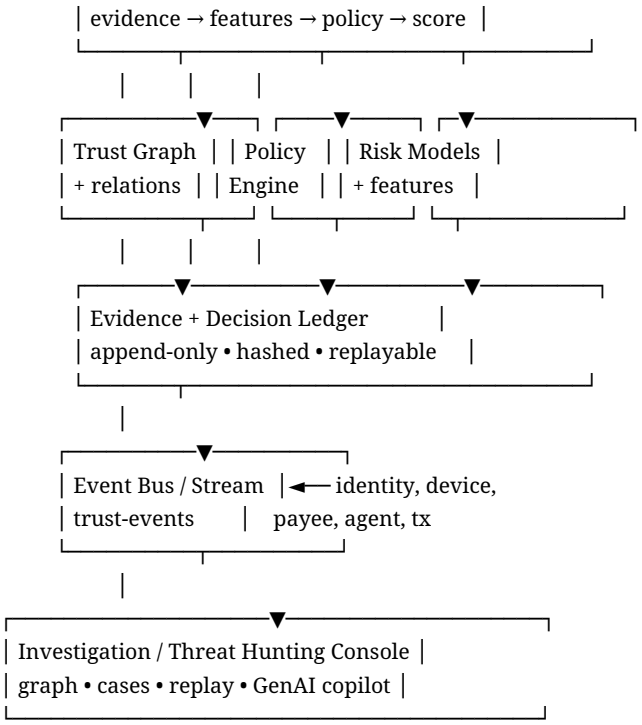
Evidence	Signed/raw/derived facts used by decision	signature verification, feature snapshot
Case	Investigation container	cluster of related events and decisions

3. Scope: build v1, v1.5 and v2

Capability	V1 — build now	V1.5	V2+
Decision API	Yes	Harden	Multi-region active/active
Rules/policy engine	Yes	Policy simulator	Customer self-service policy marketplace
Trust events	Yes	Streaming enrichment	Cross-tenant/privacy-preserving federation
Trust graph	Yes, core relationships	Graph features + ring detection	Graph ML / GNN
Risk models	Heuristics + simple ML	Supervised models + calibration	Online learning / advanced sequence models
AP2 adapter	Verify + map mandates	Full issuing/merchant flows	Additional mandate ecosystems
Visa TAP adapter	Signature verification	Key lifecycle automation	Broader protocol adapters
Agent authorization	Native bounded mandate	Delegation chains	Multi-agent delegation networks
Investigation UI	Yes	Case workflow + collaboration	Enterprise SOC integrations
GenAI investigator	Read-only, evidence-grounded	Rule suggestion + hunt planning	Controlled semi-autonomous hunting
Decision replay	Yes	Counterfactual replay	Historical simulation farm
Threat intel	Manual/watchlist ingest	External feeds	Federated consortium intelligence
Payments execution	No	No	Only via customer-owned adapters; THAAP remains control plane

4. Reference architecture





4.1 Implementation topology for the first build

Use a modular monolith for the synchronous control plane, a separate Python model service, and an event bus. This yields production-like separation without creating operational overhead before the product has traffic.

Component	Recommended first implementation
Core API + orchestration	Java 21, Spring Boot 3.x, Gradle
Policy	OPA/Rego embedded or sidecar; version all bundles
Model service	Python 3.12+, FastAPI, scikit-learn/XGBoost-compatible interface
Primary DB	PostgreSQL 16+; JSONB for extensible evidence
Graph	Neo4j Community for local/MVP; adapter interface for Amazon Neptune later
Cache / online features	Redis
Events	Redpanda locally; Kafka/MSK-compatible API
Object/evidence store	MinIO locally; S3 in AWS
Search	OpenSearch optional in V1; PostgreSQL FTS acceptable first
UI	Next.js + TypeScript + React; Cytoscape.js for graph exploration
Auth	OIDC/OAuth2; Keycloak locally; enterprise IdP/Cognito-compatible adapter
Telemetry	OpenTelemetry + Prometheus + Grafana; structured JSON logs
Infra	Docker Compose local; Terraform + EKS/ECS choice for AWS deployment

5. Decision engine — the heart of THAAP

5.1 Decision pipeline

10. Authenticate caller and validate request schema/idempotency key.
11. Canonicalize actor, agent, counterparty, instrument, instruction and transaction identifiers.
12. Verify cryptographic evidence (mandates, HTTP message signatures, JWT/JWS, certificate chain, nonce, timestamps).
13. Fetch identity/device/session risk signals and latest revocation/change events.
14. Fetch trust-graph neighborhood and graph-derived features.
15. Compute transaction, behavioral, velocity, relationship and protocol features.
16. Evaluate hard-deny rules and policy constraints.
17. Score bounded statistical models.
18. Combine policy outcome and risk score using deterministic decision matrix.
19. Generate reason codes and required controls.
20. Persist immutable evidence snapshot and decision hash before returning result.
21. Publish DecisionCreated event asynchronously for analytics, case generation and learning.

5.2 Decision semantics

Decision	Meaning	Typical action
ALLOW	Evidence and policy satisfy required confidence	Continue payment/workflow
CHALLENGE	Action may proceed only after stronger proof	Passkey, trusted callback, re-approval, mandate re-sign
REVIEW	Do not execute automatically; route to human investigator	Create case and hold action
DENY	Hard policy violation or unacceptable risk	Block action and emit high-severity event

5.3 Score composition

Do not pretend a single “AI score” is truth. Maintain interpretable component scores and a deterministic final policy. A suggested initial structure:

```
risk_score = clamp(0, 1000,
    0.24 * identity_risk
  + 0.16 * device_session_risk
  + 0.22 * counterparty_risk
  + 0.16 * behavioral_risk
  + 0.12 * graph_risk
  + 0.10 * transaction_risk
)

trust_score = 1 - risk_score/1000

Hard policy / cryptographic failures override numeric score.
```

The weights above are bootstrap defaults for engineering only, not production-calibrated fraud weights. Store them in versioned configuration and replace them with calibrated coefficients once labeled data exists.

5.4 Example decision matrix

Condition	Result

Invalid/expired/replayed mandate or signature	DENY
Agent exceeds explicit amount/merchant/category/time constraint	DENY
Known compromised identity/session signal	DENY or CHALLENGE per customer policy
New beneficiary + payment within short window + high amount	CHALLENGE/REVIEW
High graph risk to mule/shared-account cluster	REVIEW or DENY
Low risk + known relationship + no trust-event anomalies	ALLOW
Medium risk but passkey/re-approval successfully completed	Re-evaluate; may ALLOW

6. Trust Event Ledger

6.1 Why this must exist

The ledger captures the events that created or changed trust before money moved. This directly addresses the J.P. Morgan problem: a payment can look legitimate while the supplier identity, payee account or instruction was compromised earlier. [R1]

6.2 Event schema

```
TrustEvent {
  event_id: UUID
  tenant_id: UUID
  event_type: string
  occurred_at: timestamp
  observed_at: timestamp
  subject_refs: [{type, id}]
  actor_ref: {type, id}?
  source: string
  source_assurance: LOW | MEDIUM | HIGH | CRYPTOGRAPHIC
  attributes: object
  evidence_refs: [UUID]
  correlation_id: string?
  prev_event_hash: string?
  event_hash: string
}
```

6.3 Minimum event taxonomy

- IDENTITY_AUTHENTICATED, AUTHENTICATION_FAILED, PASSKEY_ASSERTED, CREDENTIAL_CHANGED, SESSION_REVOKED.
- DEVICE_FIRST_SEEN, DEVICE_RISK_CHANGED, DEVICE_COMPLIANCE_CHANGED.
- PAYEE_CREATED, PAYEE_BANK_ACCOUNT_CHANGED, PAYEE_VERIFIED, PAYEE_VERIFICATION_FAILED.
- INVOICE_CREATED, INVOICE_CHANGED, PAYMENT_INSTRUCTED, PAYMENT_APPROVED, PAYMENT_RELEASED.
- AGENT_REGISTERED, AGENT_KEY_ROTATED, AGENT_MANDATE_ISSUED, AGENT_MANDATE_REVOKED, AGENT_TOOL_CALL.
- POLICY_CHANGED, MODEL_CHANGED, WATCHLIST_MATCH, CASE_OPENED, CASE_DISPOSITIONED.

7. Trust Graph

7.1 Nodes and edges

Node	Important edges
Actor	USES_DEVICE, MEMBER_OF_ORG, OWNS_ACCOUNT, DELEGATES_TO_AGENT, APPROVES_INSTRUCTION
Agent	ACTS_FOR_ACTOR, USES_KEY, CALLS_TOOL, INTERACTS_WITH_MERCHANT, PRESENTS_MANDATE
Device	OBSERVED_IN_SESSION, USED_BY_ACTOR, ASSOCIATED_WITH_IP
Counterparty	OWNS_ACCOUNT, ISSUED_INVOICE, CHANGED_ACCOUNT, RECEIVES_PAYMENT
FinancialAccount	BELONGS_TO, RECEIVES_FROM, SHARED_WITH, PREVIOUSLY_SEEN_WITH
Instruction	CREATED_BY, APPROVED_BY, REFERENCES_INVOICE, TARGETS_ACCOUNT
Transaction	RESULTS_FROM_INSTRUCTION, SENDS_TO, INITIATED_BY, AUTHORIZED_BY_MANDATE

7.2 V1 graph features

- payee_account_age_hours
- actor_payee_relationship_age_days
- device_actor_first_seen_hours
- number_of_distinct_actors_using_device_30d
- number_of_counterparties_sharing_destination_account_30d
- hops_to_known_bad_entity
- new_bank_account_then_payment_minutes
- approval_chain_deviation_count
- agent_to_merchant_first_seen
- agent_key_age_hours
- connected_high_risk_nodes_2hop

7.3 Graph hunt queries to implement

1. Find all transactions within 2 hops of a known mule account.
2. Find devices used by >N actors across tenants where privacy policy permits.
3. Find payees that changed account details shortly before receiving large payments.
4. Find agents sharing keys, device identities or unusual merchant clusters.
5. Find approval paths that differ from historical paths for the same organization/payee.

8. Agent identity, intent and authorization

8.1 THAAP Native Mandate

Create a protocol-neutral THAAP mandate so the platform can enforce delegated authority even when AP2 is not present. Use JWS/SD-JWT-compatible signing and keep the canonical payload deterministic.

```
{
  "mandate_id": "mnd_...",
  "issuer": "user-or-org-id",
```

```
  "delegate_agent": "agt_...",
  "purpose": "PROCURE_OFFICE_SUPPLIES",
  "constraints": {
    "max_amount": {"value": 50000, "currency": "INR"},
    "merchant_ids": ["..."],
    "mcc_allow": ["5111"],
    "country_allow": ["IN"],
    "max_transactions": 5,
    "velocity_window_seconds": 86400
  },
  "not_before": "...",
  "expires_at": "...",
  "nonce": "...",
  "agent_public_key_thumbprint": "...",
  "delegation_depth": 0
}
```

8.2 AP2 adapter

- Parse and validate exact mandate schema version.
- Validate signatures, issuer/trust anchors, exp/nbf/aud/nonce and replay status.
- Validate checkout_hash / payment binding and applicable constraints.
- Normalize open/closed mandate evidence into THAAP AuthorizationEvidence.
- Persist mandate + receipt hashes for dispute replay.
- Return AP2-specific reason codes without leaking sensitive internal scoring.

8.3 Visa Trusted Agent adapter

- Validate HTTP Message Signatures and required signed components.
- Resolve approved agent key material through configured trust store.
- Bind signature verification result to request body digest, timestamp and nonce.
- Normalize verified agent ID and commerce intent into THAAP AgentEvidence.
- Never equate “Visa-approved/cryptographically valid agent” with “safe transaction”; continue full THAAP risk evaluation. [R4]

8.4 Delegation safety rules

- Default delegation_depth = 0; enable sub-delegation only by explicit policy.
- Child mandate can only narrow, never widen, parent constraints.
- Every delegation has independent expiry, nonce and key binding.
- Revocation of parent invalidates descendants.
- All mandate verification happens in deterministic code, consistent with AP2’s security model. [R3]

9. B2B trust controls

9.1 Beneficiary-change firewall

This should be the first flagship workflow because it demonstrates THAAP’s differentiation from transaction-only scoring.

22. Capture PAYEE_BANK_ACCOUNT_CHANGED event with who/what/when/source evidence.
23. Mark beneficiary as cooling-down until independently verified or timer expires per policy.
24. For payments during cooling-down, require out-of-band verification using a previously trusted channel; never verify using contact details supplied in the same change request.

25. Compute relationship history, amount deviation, device/session risk, communication-channel novelty and graph risk.
26. Return CHALLENGE or REVIEW even if normal AP approval has completed.
27. Persist verifier identity and verification evidence so future transactions can receive higher trust.

9.2 Deepfake-resistant approval pattern

Do not attempt to solve deepfake detection as the primary defense. Use phishing-resistant authentication and transaction-bound confirmation. NIST recognizes cryptographic phishing-resistant methods such as WebAuthn; use them to bind approver identity to the exact payment/mandate details. [R5]

10. Policy engine

10.1 Policy domains

- Identity assurance requirements by amount/action.
- Agent mandate constraints.
- Beneficiary change cooling periods.
- Allowed/blocked countries, currencies, MCCs, merchants and payees.
- Velocity and aggregate spend.
- Segregation-of-duties / maker-checker rules.
- Graph-risk thresholds.
- Required step-up method.
- Customer-specific hard deny lists and watchlists.
- Fail-open/fail-closed behavior by action criticality.

10.2 Example Rego-like rule

```
deny["NEW_PAYEE_HIGH_VALUE"] if {
  input.transaction.amount >= 1000000
  input.payee.account_age_hours < 24
  not input.payee.independent_verification_passed
}

challenge["AGENT_REQUIRES_STEP_UP"] if {
  input.actor.kind == "AGENT"
  input.risk_score >= 450
  input.risk_score < 700
}
```

10.3 Policy safety

- Every policy bundle is immutable and content-addressed.
- Changes require dual approval in production.
- Support dry-run/shadow evaluation before activation.
- Decision records store policy bundle hash and rule IDs fired.
- Provide a counterfactual simulator: “What would policy v17 have done on last week’s transactions?”

11. Risk and ML layer

11.1 V1 models

Start with transparent models. A labeled-data-poor startup gains more from good event design and feature quality than a complex neural model.

Model	Initial technique	Output
Transaction anomaly	Robust z-scores / Isolation Forest	0–1000 anomaly risk
Behavior profile	Per-actor/payee baselines + rules	deviation features
Graph risk	Heuristic weighted neighborhood score	0–1000 graph risk
Supervised fraud model	Logistic regression / gradient boosting when labels exist	calibrated probability
Agent behavior	Constraint violation + velocity + novelty	0–1000 agent risk

11.2 Feature provenance

Every feature value must carry provenance: source events, computation timestamp, feature definition version and freshness. Replay must reconstruct the exact feature vector used at decision time.

11.3 Model governance

- Model registry with version, training dataset hash, metrics, owner and approval state.
- Shadow deployment before serving decisions.
- Drift monitoring for feature distributions and score distributions.
- Calibration metrics, false-positive rate, recall, precision, review rate and prevented-loss proxy.
- No automatic online model update directly to production; training and promotion are controlled workflows.

12. GenAI investigation copilot

12.1 What it should do

- Summarize a case from signed decision/evidence records.
- Explain reason codes in analyst-friendly language.
- Generate graph-hunt queries from an investigator question, then require deterministic query execution.
- Compare a suspicious transaction to prior trusted behavior.
- Propose new rules from confirmed fraud cases for analyst review.
- Create an incident timeline.
- Generate a customer-facing explanation that excludes protected/internal model details.

12.2 What it must not do

- Directly call a payment execution tool.
- Override DENY/CHALLENGE/REVIEW decisions.
- Write policy to production without review.
- Treat retrieved text, email, merchant content, MCP output or agent memory as trusted instructions.
- Modify immutable evidence or decision history.

12.3 Copilot security

Implement the copilot as a read-mostly principal with an explicit tool allowlist. All retrieved content is untrusted data. Separate system policy from case content. Log every tool call, arguments, result hash and

model/version. The design should be tested against OWASP agent goal hijacking, tool misuse, privilege abuse, memory poisoning and insecure inter-agent communication. [R7][R8]

13. Decision ledger and replay

13.1 Decision record

```
DecisionRecord {
  decision_id, replay_id, tenant_id,
  request_hash,
  normalized_request_snapshot,
  evidence_refs[], evidence_merkle_root,
  feature_vector_snapshot,
  policy_bundle_hash, fired_rule_ids[],
  model_scores[], model_versions[],
  result, reason_codes[], required_controls[],
  created_at, processing_ms,
  previous_decision_id?,
  record_hash
}
```

13.2 Replay modes

Mode	Purpose
Exact replay	Reconstruct original outcome from immutable snapshots; used for audit/dispute.
Current-policy replay	Run old event against today's policy/model to measure improvement/regression.
Counterfactual replay	Change one condition (e.g., passkey asserted, payee verified) to explain what control would have changed outcome.
Batch simulation	Evaluate proposed policy/model against historical corpus before promotion.

14. APIs to implement

14.1 Core decision endpoint

```
POST /v1/decisions
Idempotency-Key: <uuid>
Authorization: Bearer <token>

{
  "action": "PAYMENT_AUTHORIZE",
  "actor": {...},
  "agent": {... optional ...},
  "device": {...},
  "counterparty": {...},
  "instruction": {...},
  "transaction": {...},
  "authorization_evidence": [...],
  "context": {...}
```

```
}

200 {
  "decision_id": "dec_...",
  "result": "CHALLENGE",
  "risk_score": 612,
  "trust_score": 0.388,
  "reason_codes": ["PAYEE_CHANGED_RECENTLY","NEW_DEVICE"],
  "required_controls": ["PASSKEY_TRANSACTION_CONFIRMATION","PAYEE_CALLBACK"],
  "expires_at": "...",
  "replay_id": "rpl_..."
}
```

14.2 Additional endpoints

Endpoint	Purpose
POST /v1/trust-events	Ingest trust-changing event
POST /v1/mandates	Create THAAP native mandate
POST /v1/mandates/{id}/revoke	Revoke mandate and descendants
POST /v1/protocols/ap2/verify	Verify and normalize AP2 evidence
POST /v1/protocols/visa-tap/verify	Verify Trusted Agent request evidence
GET /v1/decisions/{id}	Retrieve decision summary
POST /v1/replays/{id}	Run authorized replay mode
GET /v1/graph/entities/{type}/{id}	Get bounded graph neighborhood
POST /v1/cases	Create investigation case
POST /v1/cases/{id}/copilot	Grounded investigation query
POST /v1/policies/simulate	Dry-run policy bundle against samples

15. Database schema — minimum viable

Table	Critical columns
tenants	id, name, config, created_at
actors	id, tenant_id, type, external_ref, status, attributes
agents	id, tenant_id, provider, public_key_ref, status, attributes
devices	id, tenant_id, fingerprint_hash, first_seen, last_seen, risk_state
counterparties	id, tenant_id, type, external_ref, status
financial_accounts	id, tenant_id, owner_ref, tokenized_identifier, status, verified_at
instructions	id, tenant_id, type, payload_hash, created_by, status
transactions	id, tenant_id, instruction_id, amount, currency, status, external_ref
mandates	id, tenant_id, issuer_ref, agent_ref, constraints_json, signature_ref, status, expiry
trust_events	id, tenant_id, type, occurred_at, subjects_json, attributes_json, event_hash
evidence	id, tenant_id, type, object_uri, content_hash, assurance
decisions	id, tenant_id, result, scores_json, reason_codes, policy_hash, evidence_root, record_hash

cases	id, tenant_id, status, severity, disposition, owner
watchlist_entries	id, tenant_id, entity_type, entity_hash, reason, effective_from, effective_to
model_registry	id, name, version, artifact_uri, artifact_hash, status, metrics_json
policy_registry	id, version, bundle_uri, bundle_hash, status, approved_by

16. Event contracts

16.1 Kafka topics

thaap.trust-events.v1
thaap.decisions.v1
thaap.cases.v1
thaap.feedback.v1
thaap.model-events.v1
thaap.policy-events.v1
thaap.security-events.v1

16.2 Event guarantees

- At-least-once delivery with idempotent consumers.
- event_id is globally unique; producer includes schema_version.
- Partition by tenant_id + stable entity key where ordering matters.
- PII minimization: use tokens/hashes in event payloads; resolve sensitive data only in authorized services.
- Dead-letter topic with alerting and replay tooling.
- Outbox pattern for DB-to-Kafka publication so decisions and events cannot diverge.

17. Threat model and security controls

Threat	Required THAAP control
Stolen user credentials	Passkeys/phishing-resistant step-up, device/session signals, continuous revocation events
Business email compromise	Independent beneficiary verification, cooling period, instruction-source risk
Deepfake approver	Transaction-bound cryptographic confirmation; voice/video only supplementary
Agent goal hijack	Constraints enforced outside LLM, signed mandates, explicit tool allowlist
Tool misuse / excessive agency	Least privilege, per-tool scope, spend/velocity limits, deny-by-default
Memory poisoning	Treat memory as untrusted, provenance/TTL, no memory-to-policy path
Agent identity spoofing	Signed agent requests, trusted key registry, key rotation/revocation
Replay attack	Nonce, exp/nbf, one-time mandate use where required, idempotency
Evidence tampering	Content hashes, append-only object retention, KMS-signed roots
Insider policy manipulation	Dual approval, immutable bundle versions, audit trail

Model compromise	Artifact hashes/signatures, controlled registry promotion, isolated serving
Cascading agent failure	Circuit breakers, transaction/velocity caps, human escalation, blast-radius limits

17.1 Data security

- Tokenize account/card identifiers; never use raw PAN as a graph key.
- Encrypt in transit and at rest; KMS-managed keys in AWS.
- Column/field-level protection for sensitive identity attributes.
- Separate customer tenants logically from day one; prepare for stronger physical isolation tiers later.
- Fine-grained RBAC/ABAC for investigators, policy authors and admins.
- Immutable audit events for reads of sensitive cases as well as writes.
- Configure retention by data class; allow evidence legal-hold without retaining unnecessary PII.

17.2 Compliance baseline

Design the product so PCI DSS scope is minimized by using tokens and external credential providers rather than storing cardholder data. Maintain a control mapping for PCI DSS v4.0.1 where applicable. For AI governance, map model and copilot controls to NIST AI RMF / GenAI Profile and OWASP agentic risks. [R9][R10]

18. Observability and SLOs

Metric/SLO	Initial target
Decision API availability	99.9% prototype target; design path to 99.99% for financial production
Decision p95 latency	< 100 ms without external challenge; < 200 ms with graph/model lookups
Decision p99 latency	< 300 ms target
Policy evaluation	< 15 ms p95
Graph feature fetch	< 40 ms p95 for bounded neighborhood features
Model scoring	< 20 ms p95 per synchronous model
Replay completeness	100% of decisions must have enough stored evidence metadata for exact replay
Audit integrity	100% decision records carry record_hash + evidence_root
Event processing lag	< 5 seconds for trust-event derived state in normal operation

These are engineering targets for the prototype and should be validated under load; they are not promises to customers.

19. Investigator console

19.1 Pages

28. Decision feed — filter by result, risk, reason code, tenant, actor, agent, payee and time.
29. Decision detail — timeline, score components, fired policies, mandate verification and evidence.
30. Trust graph explorer — bounded neighborhood with risk overlays and time slicing.

31. Case workspace — notes, entities, linked decisions, status, disposition and feedback label.
32. Replay lab — exact/current/counterfactual replay and side-by-side result diff.
33. Policy lab — edit test policy, run against golden dataset, inspect impacted decisions.
34. Model health — distribution/drift/latency/quality metrics.
35. Agent registry — identities, keys, mandates, revocations, spend/velocity state.

20. AWS production mapping

Logical component	AWS target
API / core	EKS or ECS Fargate behind ALB/API Gateway
PostgreSQL	Aurora PostgreSQL
Redis	ElastiCache for Redis/Valkey-compatible service
Kafka	Amazon MSK
Graph	Amazon Neptune
Object evidence	S3 + versioning/Object Lock where appropriate
Search	Amazon OpenSearch Service
Keys/signing	AWS KMS / CloudHSM where assurance requires
Secrets	AWS Secrets Manager
Identity	Enterprise OIDC; Cognito optional for SaaS control plane
Telemetry	OpenTelemetry collectors + CloudWatch/Prometheus/Grafana stack
WAF / edge	AWS WAF + Shield where appropriate
Deployment	Terraform + GitHub Actions or enterprise CI/CD

21. Repository structure for Codex

```
thaap/
├── README.md
├── docs/
│   ├── architecture.md
│   ├── threat-model.md
│   ├── api.md
│   ├── data-model.md
│   └── runbooks/
├── apps/
│   ├── core-api/            # Java Spring Boot modular monolith
│   ├── model-service/       # Python FastAPI
│   └── console/             # Next.js TypeScript
├── modules/
│   ├── decision/
│   ├── policy/
│   ├── evidence/
│   ├── trust-events/
│   ├── graph/
│   ├── mandates/
│   ├── protocol-ap2/
│   ├── protocol-visa-tap/
│   ├── cases/
```

```
│   └── audit/
├── contracts/
│   ├── openapi/
│   ├── json-schema/
│   └── events/
├── infra/
│   ├── docker-compose.yml
│   └── terraform/
├── policies/
│   ├── base/
│   └── tests/
├── test-data/
│   ├── golden-decisions/
│   └── fraud-scenarios/
└── scripts/
    ├── seed-demo.sh
    ├── run-e2e.sh
    └── replay-scenario.sh
```

22. Delivery plan

Phase 0 — foundation

- Monorepo, Docker Compose, CI, code quality, dependency scanning.
- OIDC auth, tenant model, migrations, OpenAPI-first contracts.
- Postgres + Redis + Redpanda + Neo4j local stack.
- OpenTelemetry tracing and correlation IDs.

Phase 1 — deterministic trust decision MVP

- TrustEvent ingestion and append-only ledger.
- Decision API and normalized object model.
- Policy engine and hard rules.
- Basic feature engine and risk-score composition.
- Decision ledger + exact replay.
- Demo scenario: new supplier bank account → large payment → CHALLENGE/REVIEW.

Phase 2 — trust graph and investigations

- Graph projection and core graph features.
- Graph explorer UI, decision feed, case workspace.
- Known-bad/watchlist matching.
- Fraud-ring demo with shared destination account/device.

Phase 3 — agentic authorization

- THAAP Native Mandate issuance/verification/revocation.
- AP2 verifier adapter and evidence normalization.
- Visa TAP-compatible message signature verifier.
- Agent registry, public keys, spend limits, velocity and replay protection.
- Human-present and human-not-present demo flows.

Phase 4 — ML + GenAI investigator

- Feature snapshots and model registry.

- Baseline anomaly/supervised models with offline evaluation.
- Read-only grounded investigator copilot.
- Rule recommendation workflow requiring human approval.
- Agentic red-team test suite.

Phase 5 — production hardening

- Load/failure testing, chaos scenarios, DR, backups and retention.
- Policy/model promotion workflows and dual approval.
- Security review, threat-model validation, SBOM/AIBOM, dependency signing.
- AWS Terraform target and performance benchmark suite.
- Privacy/PCI/AI governance control mappings.

23. Mandatory end-to-end demo scenarios

Scenario	Expected result
Normal recurring supplier payment from known user/device/account	ALLOW
Supplier bank account changed 30 min ago; high-value payment	CHALLENGE or REVIEW
Same as above + independent verified callback + passkey	Re-evaluate; ALLOW if residual risk acceptable
Valid AP2 agent exceeds max amount in signed mandate	DENY
Valid agent + valid mandate + unusual velocity spike	CHALLENGE/REVIEW
Expired/replayed mandate	DENY
Credential/session revocation arrives after login but before payment	DENY/CHALLENGE
Destination account connected to known mule within 2 hops	REVIEW/DENY
Prompt injection inside invoice note tries to tell copilot to approve	No effect on decision; copilot treats content as untrusted
Replay old decision after policy change	Exact replay equals old result; current-policy replay may differ with diff shown

24. Testing requirements

24.1 Unit/property tests

- Mandate constraint narrowing and expiry.
- Nonce/replay rejection.
- Money/currency arithmetic uses decimal types, never binary floating point.
- Policy determinism: same normalized input + same policy/model versions => same result.
- Hash canonicalization stability.
- Graph feature boundary conditions.
- PII redaction in logs and events.

24.2 Security tests

- JWT/JWS algorithm confusion, key substitution, expired/nbf/audience mismatch.
- HTTP Message Signature body/header mutation.
- Prompt injection through invoice text, merchant content, case notes and retrieved documents.

- MCP/tool argument injection.
- Privilege escalation from investigator/copilot identity.
- Policy-bundle tampering and unauthorized promotion.
- Replay/race conditions around one-time mandates.
- Tenant isolation tests on every repository/query path.

24.3 Golden fraud corpus

Create deterministic JSON scenarios under test-data/fraud-scenarios. Every scenario contains events, request, expected decision, mandatory reason codes and optional expected graph relations. CI must run the corpus on every policy/decision-engine change.

25. Acceptance criteria for the first “real” THAAP

36. A developer can run the full system locally with one command.
37. The system can ingest trust events, construct relationships and return ALLOW/CHALLENGE/REVIEW/DENY.
38. Every decision exposes stable machine-readable reason codes.
39. Every decision is exactly replayable from persisted evidence/feature/policy/model metadata.
40. A beneficiary-change fraud scenario is detected before funds move.
41. An AP2-style agent transaction can be cryptographically verified and denied when it violates signed constraints.
42. An agent request cannot bypass policy simply because its identity/signature is valid.
43. The investigation UI shows a timeline and graph neighborhood for a decision.
44. The copilot can explain a case but cannot mutate policy, evidence or payment state.
45. Automated tests prove replay protection, tenant isolation, signature validation and prompt-injection containment.
46. Load test demonstrates the prototype latency budget under a documented workload.
47. Documentation includes threat model, API spec, local runbook and production migration notes.

26. Product moat to intentionally build

THAAP’s defensibility should come from evidence quality and accumulated trust intelligence, not from an LLM wrapper. Build these assets deliberately:

- A canonical cross-protocol authorization/evidence model.
- Trust Event Ledger spanning pre-payment workflow changes and transaction execution.
- Temporal Trust Graph with reusable graph features and hunt patterns.
- Decision Replay that proves exactly why a payment was allowed or blocked.
- Agent behavior profiles and mandate-constraint history.
- Customer policy simulation and reason-code feedback loops.
- Confirmed-case feedback that improves rules/features/models while preserving governance.
- Protocol adapters so THAAP becomes the common control layer across AP2, Visa TAP and future standards.

27. Explicit decisions for Codex — do not improvise these

Decision	Required choice
Architecture	Modular monolith core API + separate model service + web console + event bus.
Core language	Java 21 / Spring Boot for synchronous decision control plane.
ML language	Python 3.12+ / FastAPI.

Frontend	Next.js + TypeScript.
Primary database	PostgreSQL.
Graph	Neo4j behind GraphRepository interface; make Neptune migration possible.
Cache	Redis.
Events	Kafka API / Redpanda local.
Policy	OPA/Rego with immutable versioned bundles.
Evidence	Object store + hashes; metadata in PostgreSQL.
Money movement	No execution connector in initial product.
LLM permissions	Read-only investigation tools; no decision override or payment tool.
Multi-tenancy	tenant_id mandatory in all domain data and access paths.
API design	OpenAPI-first, idempotency keys on mutating endpoints.
Security	OIDC, RBAC/ABAC, signed mandates, nonce/replay controls, PII-safe logs.
Observability	OpenTelemetry traces/metrics/log correlation from day one.

28. Master build instruction for Codex

Give Codex this document as the authoritative product specification, then use the following instruction. Do not ask it to “build THAAP” in one giant unreviewed change; require milestone commits and passing acceptance tests at each phase.

> You are implementing THAAP from the attached THAAP Implementation Blueprint v1.0.
> Treat the document as the product and architecture specification.
>
> Rules:
> 1. Implement Phase 0 and Phase 1 first. Do not skip ahead.
> 2. Keep the core decision path deterministic. No LLM may output or override ALLOW/DENY/CHALLENGE/REVIEW.
> 3. Use the exact repository structure unless a build-system constraint requires a documented deviation.
> 4. Define OpenAPI and JSON/event schemas before endpoint implementation.
> 5. Every mutating endpoint must be idempotent where semantically applicable.
> 6. Every domain query must be tenant-scoped.
> 7. Money uses decimal types and explicit ISO currency codes.
> 8. Every decision persists evidence references, feature snapshot, policy version, model version(s), reason codes and record hash.
> 9. Exact replay must be a first-class automated test.
> 10. Add unit, integration, security and end-to-end tests for every milestone.
> 11. Create seeded demo scenarios described in Section 23.
> 12. Add Docker Compose so the complete development stack starts locally.
> 13. Add README/runbooks with commands to build, test, seed, run and replay.
> 14. Do not introduce a payment-execution capability.
> 15. Stop at the end of each phase, run all tests, fix failures, summarize implementation, then continue.
>
> Definition of done: all Section 25 acceptance criteria pass.

29. Recommended build order inside Phase 1

48. Domain IDs/value objects and tenant boundary.
49. Database migrations and repositories.

50. TrustEvent schema + ingestion API + event hashing.
51. Canonical DecisionRequest schema.
52. Evidence abstraction and content-hash storage.
53. FeatureProvider interface and deterministic baseline features.
54. PolicyDecision interface + OPA integration.
55. RiskScoreComposer.
56. DecisionOrchestrator.
57. Decision ledger and record hashing.
58. Exact replay service.
59. Reason-code catalog.
60. Beneficiary-change flagship policy and scenario.
61. Integration tests + golden corpus.
62. Minimal console decision list/detail.

30. Success metrics

Category	Metric
Risk quality	Fraud recall, false-positive rate, precision, review rate, prevented-loss proxy
Trust quality	Challenge success rate, beneficiary verification success, mandate violation rate
Agentic	Invalid/replayed mandate rate, agent constraint breach rate, human-escalation rate
Operations	Decision p95/p99, event lag, replay success, model/policy rollback time
Investigation	Mean time to understand, time to linked-entity discovery, case closure time
Product	Time to integrate, policies per tenant, decision volume, retained customers

31. Research references

[R1] J.P. Morgan — AI-powered B2B fraud is forcing companies to rethink trust (14 Sep 2026)
https://www.jpmorgan.com/insights/fraud/fraud-prevention/ai-powered-b2b-fraud-is-forcing-companies-to-rethink-trust

[R2] Mastercard — Trust in agentic commerce / Agent Pay Trust Framework (30 Sep 2026)
https://www.mastercard.com/global/en/news-and-trends/Insights/2026/mastercard-agentic-commerce-trust-white-paper.html

[R3] Agent Payments Protocol (AP2) — Agentic Payment Protocol v0.2 specification (2026)
https://ap2-protocol.org/ap2/specification/

[R4] Visa Developer — Trusted Agent Protocol — getting started and specifications (2026)
https://developer.visa.com/capabilities/trusted-agent-protocol/docs-getting-started

[R5] NIST — SP 800-63-4 Digital Identity Guidelines (2025)
https://csrc.nist.gov/pubs/sp/800/63/4/final

[R6] OpenID Foundation — Shared Signals Framework / CAEP specifications (Current)
https://openid.net/wg/sharedsignals/specifications/

[R7] OWASP GenAI Security Project — Top 10 for Agentic Applications 2026 (2025-2026)
https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[R8] OWASP GenAI Security Project — AIUC-1 crosswalk to OWASP Top 10 for Agentic Applications (25 May 2026)
https://genai.owasp.org/resource/aiuc-1-crosswalks-owasp-top-10-for-agentic-applications/

[R9] PCI Security Standards Council — PCI DSS v4.0.1 document library (Current)
https://www.pcisecuritystandards.org/document_library/

[R10] NIST — AI Risk Management Framework + Generative AI Profile (2024-2026)
https://www.nist.gov/itl/ai-risk-management-framework

[R11] OpenAI — Agentic Commerce Protocol developer documentation (2025-2026)
https://developers.openai.com/commerce

[R12] Google Cloud — Powering AI commerce with the Agent Payments Protocol (AP2) (16 Sep 2025)
https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol

[R13] Stripe — Radar: primer on machine learning for fraud detection (15 Dec 2021)
https://stripe.com/guides/primer-on-machine-learning-for-fraud-protection

[R14] Mastercard — Mastercard launches Agent Pay for Machines (10 Jun 2026)
https://www.mastercard.com/us/en/news-and-trends/press/2026/june/mastercard-launches-agent-pay-for-machines.html

Research note

This blueprint intentionally favors primary/standards sources for product and security direction. Some standards and commercial protocols are evolving quickly; adapters should therefore be versioned, isolated behind interfaces, and covered by conformance tests. The document is an engineering/product blueprint, not legal advice or a certification claim.